Table of Contents

3D World — Project Guide **2**
- Table of Contents . . . 2
- 1. Architecture Overview . . . 2
  - What This Project Is . . . 2
  - Tech Stack . . . 3
  - High-Level Architecture . . . 3
  - Source File Map . . . 3
- 2. The 3D World Engine . . . 4
  - Rendering . . . 4
  - Physics . . . 5
  - Player Controller . . . 5
  - Game Loop . . . 6
- 3. Level Loading . . . 6
  - GLB Level Mode . . . 6
  - Procedural Level Mode . . . 6
  - Switching Between Modes . . . 7
- 4. The Agent Bridge — How Claude Code Connects . . . 7
  - The Communication Chain . . . 7
  - Enabling Agent Mode . . . 8
  - Key Implementation Details . . . 8
- 5. API Reference — Perception . . . 8
  - Get State . . . 8
  - Screenshot . . . 8
  - Panorama . . . 9
  - Surroundings . . . 9
  - Raycast . . . 10
  - Screen Raycast . . . 10
  - Detect (DINO Object Detection) . . . 11
- 6. API Reference — Actions . . . 12
  - Move . . . 12
  - Rotate . . . 13
  - Jump . . . 13
  - Jump + Move . . . 13
  - Move To . . . 13
  - Look At . . . 13
  - Sequence . . . 14
  - Teleport . . . 14
  - Undo . . . 14
  - History . . . 14
  - Stop . . . 14
- 7. DINO Object Detection . . . 14
  - What It Is . . . 14
  - How It Works . . . 15
  - The Detection Server . . . 15
  - Label Format . . . 15
  - Annotated Output . . . 15

The Detection-to-Navigation Workflow . . . 16
8. Navigation Patterns . . . 16
Coordinate System . . . 16
Room Entry Protocol . . . 16
Doorway Navigation (Detect-Aim-Walk) . . . 17
Wall Hugging . . . 17
Backtracking When Stuck . . . 17
Error Handling . . . 17
9. Getting Started . . . 18
Prerequisites . . . 18
Install Dependencies . . . 18
Start the Servers . . . 18
Open the Browser . . . 19
First Commands . . . 19
Quick Exploration Walkthrough . . . 19
Appendix: Coordinate Quick Reference . . . 20
Appendix: Complete API Endpoint Table . . . 20

3D World — Project Guide

A first-person 3D environment with physics simulation and an HTTP API that lets AI agents (like Claude Code) explore the world through screenshots, raycasting, and object detection.

Built with Three.js, Rapier3D physics, TypeScript, and Vite.

Table of Contents

1. Architecture Overview
2. The 3D World Engine
3. Level Loading
4. The Agent Bridge — How Claude Code Connects
5. API Reference — Perception
6. API Reference — Actions
7. DINO Object Detection
8. Navigation Patterns
9. Getting Started

---

1. Architecture Overview

What This Project Is

This is a browser-based first-person 3D world where:

- A **human player** can walk around with WASD + mouse, or
- An **AI agent** (Claude Code) can control the character via HTTP commands and perceive the world through screenshots and sensor data.

The world supports loading GLB/glTF 3D models with full physics collision, meaning the agent navigates a real 3D environment — not a grid or text adventure.

Tech Stack

Component	Technology	Purpose
3D Rendering	Three.js v0.182	WebGL scene, camera, lighting, shadows
Physics	Rapier3D v0.19 (WASM)	Gravity, collision, character controller
Language	TypeScript 5.7 (strict)	Type-safe source code
Bundler	Vite 6.0	Dev server, HMR, HTTP middleware
Agent Comms	WebSocket + HTTP	Bridge between curl and browser
Object Detection	Grounding DINO Tiny (Python)	Zero-shot bounding box detection

High-Level Architecture

```
+------------------+       HTTP (curl)       +-------------------+
|                  | ----------------------> |                   |
|   Claude Code    |    POST /agent/move     |   Vite Dev Server |
|   (AI Agent)     |    POST /agent/screenshot|   (agent-plugin)  |
|                  | <---------------------- |                   |
+------------------+     JSON responses      +--------+----------+
                                                      |
                                                 WebSocket
                                                      |
                                                +--------v----------+
                                                |                   |
                                                |   Browser Tab     |
                                                |   Three.js Scene  |
                                                |   Rapier Physics  |
                                                |   Agent Bridge    |
                                                |                   |
                                                +-------------------+

                      +-------------------+
                      |  detect-server.py |  (optional)
                      |  Grounding DINO   |  localhost:8190
                      |  Object Detection |
                      +-------------------+
```

Source File Map

`src/`

```
  main.ts                     Entry point, URL param parsing, RAPIER init
  Game.ts                     Main orchestrator, subsystem wiring

  core/
    Engine.ts                 Three.js renderer, scene, camera
    GameLoop.ts               Fixed-timestep loop (60 Hz)
    InputManager.ts           Keyboard/mouse state tracking
    AssetLoader.ts            GLTFLoader and TextureLoader wrappers

  player/
    FPSCamera.ts              Mouse-look with Euler rotation, pointer lock
    PlayerController.ts       WASD movement, jumping, gravity, capsule physics

  physics/
    PhysicsWorld.ts           Rapier world, character controller config
    ColliderFactory.ts        Trimesh and cuboid collider generation

  world/
    LevelLoader.ts            GLB loading and procedural room generation
    Lighting.ts               Ambient, directional, and hemisphere lights
    StarterRoomMaterials.ts   PBR texture sets for procedural level

  agent/
    types.ts                  Command/response TypeScript interfaces
    AgentController.ts        Movement/rotation state machine
    AgentSensors.ts           Raycasting, screenshots, surroundings
    AgentBridge.ts            WebSocket client in browser

agent-plugin.ts               Vite plugin: HTTP middleware + WebSocket server
detect-server.py              Python FastAPI server for DINO detection
```

---

2. The 3D World Engine

Rendering

The engine wraps Three.js with these settings:

- **WebGL Renderer** with antialiasing enabled
- **PCF Soft Shadow Maps** at 2048x2048 resolution
- **Perspective Camera** with 75-degree FOV, near=0.1, far=1000
- **Pixel ratio** clamped to 2 (prevents performance issues on high-DPI screens)

Lighting uses three layers:

Light Type	Color	Intensity	Purpose
Ambient	White	0.4	Base illumination everywhere

Light Type	Color	Intensity	Purpose
Directional	White	0.8	Sun-like shadows from (10, 20, 10)
Hemisphere	Blue sky / Dark ground	0.3	Natural sky-to-ground gradient

All meshes cast and receive shadows. The directional light's shadow camera covers a 50x50 unit area, which is enough for most level geometry.

Physics

The physics engine is **Rapier3D** (a Rust-based WASM physics library):

- **World Gravity:** 9.81 m/s^2 downward (for dynamic bodies)
- **Player Gravity:** 20 m/s^2 downward (enhanced for responsive feel)
- **Character Controller:** Kinematic position-based (not velocity-driven)
- **Character Shape:** Capsule with 0.3m radius and 0.6m half-height

Character controller features:

Feature	Value	Effect
Auto-step height	0.4m	Walks over small obstacles like curbs
Max climb slope	50 degrees	Can walk up moderate inclines
Min slide slope	30 degrees	Slides down steep slopes
Snap-to-ground	0.5m	Prevents jitter when walking down ramps

Collider generation works two ways:

- **Trimesh colliders** — for complex GLB geometry. The factory extracts vertices and triangle indices from each mesh, bakes the world transform into the vertices, and creates a triangle mesh collider. This handles arbitrary shapes: curved walls, catwalks, staircases.
- **Cuboid colliders** — for procedural box primitives. Cheaper than trimesh, used for walls, floors, and crates in the procedural level.

All level colliders share a single static rigid body for performance.

Player Controller

The first-person player controller handles:

- **Movement:** WASD keys at 8 units/second
- **Jumping:** 6 units/second impulse, enhanced gravity at 20 units/s for responsive feel
- **Eye height:** 0.7m above the physics body center
- **Camera:** Euler rotation (YXZ order) with 0.002 rad/pixel sensitivity, pitch clamped to plus/minus 90 degrees

The controller pre-allocates all scratch vectors to achieve zero per-frame allocations.

Game Loop

The game loop uses a **fixed-timestep accumulator** pattern:

1. Measure elapsed real time since last frame
2. Clamp to 0.25 seconds maximum (prevents spiral-of-death on lag)
3. Accumulate time, consuming it in fixed 1/60-second physics steps
4. Each physics step: update camera, update player controller, step Rapier world
5. Render one frame after all physics steps complete

This ensures deterministic physics regardless of display refresh rate.

---

3. Level Loading

The project supports two level modes, selected via URL parameter.

GLB Level Mode

URL: `http://localhost:5173/?level=glb`

The GLB pipeline:

1. **Load** — `AssetLoader.loadGLTF('/models/full-facility.glb')` uses Three.js GLTFLoader to parse the binary glTF file and all embedded textures.
2. **Scale** — The entire scene group is scaled by 0.0125 (1/80th) to convert from the model's internal units to game-world meters.
3. **Shadows** — Every mesh gets `castShadow = true` and `receiveShadow = true`. Meshes with names starting with "Secondary" get `alphaTest = 0.5` for transparent foliage cutouts.
4. **Physics** — `ColliderFactory.createTrimeshesFromScene()` traverses every mesh in the loaded group, clones its geometry, applies the world matrix to vertices, and creates a trimesh collider. The cloned geometry is immediately disposed to prevent GPU memory leaks.
5. **Spawn** — The player is teleported to (-13.5, -4.4, -7.5) in the scaled coordinate space.

The included model (`public/models/full-facility.glb`) is a recreation of the Facility level from GoldenEye 007. It features tiled rooms, corridors, metal catwalks with railings, a switchback staircase, and multi-level industrial architecture.

Procedural Level Mode

URL: `http://localhost:5173/?level=procedural` (or just `http://localhost:5173/`)

A 5-room facility built from box primitives with PBR textures:

Room	Dimensions	Features
Main Hall	20x20m, 6m ceiling	Crates, raised platform, brick walls
Corridor	4x12m, 3.5m ceiling	Straight passage connecting halls
Storage Hall	16x16m, 4.5m ceiling	Central hub, crate stacks, metal platform
Catwalk Room	16x12m, 8m ceiling	3m-high catwalk with ramp access, concrete pillars

Room	Dimensions	Features
Utility Room	8x8m, 3m ceiling	Stairs along wall, small crates

Each room uses PBR texture sets (Color, Normal, Roughness) for concrete, brick, metal, and wood surfaces. UV scaling is calculated per-face so textures tile at consistent density regardless of geometry size.

Doorways are constructed by splitting walls into segments around the opening with a 0.02m inset on the frame to prevent z-fighting artifacts.

Switching Between Modes

The `?level` URL parameter controls which mode is used:

```
http://localhost:5173/?level=procedural     Procedural rooms (default)
http://localhost:5173/?level=glb            GLB model
http://localhost:5173/?level=glb&agent      GLB model + agent control
```

---

4. The Agent Bridge — How Claude Code Connects

The Communication Chain

When Claude Code sends a curl command, here is what happens:

```
1. curl POST http://localhost:5173/agent/move  {"direction":"forward","distance":3}
         |
         v
2. Vite dev server HTTP middleware (agent-plugin.ts) intercepts /agent/* routes
         |
         v
3. Plugin validates parameters, generates a UUID, creates a WebSocket message
         |
         v
4. Message sent via WebSocket to the browser tab at /agent-ws
         |
         v
5. AgentBridge.ts receives the message, dispatches to AgentController or AgentSensors
         |
         v
6. Game engine executes the command (moves character, captures screenshot, etc.)
         |
         v
7. Result sent back via WebSocket to the plugin
         |
         v
8. Plugin post-processes (saves screenshots to disk, records history)
         |
```

```
         v
9. HTTP JSON response returned to curl
```

Enabling Agent Mode

Add `?agent` to the URL when opening the browser tab:

```
http://localhost:5173/?agent
http://localhost:5173/?level=glb&agent
```

This triggers the game to create the agent subsystem (AgentSensors, AgentController, AgentBridge) and hides the "click to play" UI overlay. The AgentBridge immediately connects to the WebSocket server provided by the Vite plugin.

Key Implementation Details

- **Timeout:** Each command has a 15-second timeout. If the browser does not respond, the plugin returns HTTP 504.
- **Screenshot saving:** Base64 PNG data from the browser canvas is decoded and written to `_agent-output/screenshot-{timestamp}.png` on disk. The file path is returned in the JSON response so the agent can use the Read tool to view the image.
- **Action history:** The plugin records the last 20 movement/rotation actions with position snapshots, enabling the undo feature.
- **Error mapping:** `Action in progress` maps to HTTP 409 (conflict), invalid parameters to 400, browser not connected to 503.

---

5. API Reference — Perception

All endpoints are on `http://localhost:5173`. Use `curl -s` for clean output.

Get State

Returns current position, orientation, and grounded status.

```
curl -s http://localhost:5173/agent/state
```

Response:

```
{
  "position": { "x": 0.0, "y": 1.4, "z": 0.0 },
  "orientation": { "yaw": 0, "pitch": 0 },
  "grounded": true,
  "verticalVelocity": 0
}
```

Position Y includes eye height (~1.4 when standing on ground at Y=0).

Screenshot

Captures the current view as a PNG file saved to disk.

```curl
# Current view
curl -s -X POST http://localhost:5173/agent/screenshot

# Temporary rotation without changing facing (offset from current)
curl -s -X POST http://localhost:5173/agent/screenshot \
  -H "Content-Type: application/json" \
  -d '{"yawOffset": -90}'

# Absolute angle
curl -s -X POST http://localhost:5173/agent/screenshot \
  -H "Content-Type: application/json" \
  -d '{"yaw": 180, "pitch": 0}'
```

Response:

```json
{
  "success": true,
  "path": "_agent-output/screenshot-1708300000.png",
  "position": { "x": 0, "y": 1.4, "z": 0 },
  "orientation": { "yaw": 0, "pitch": 0 }
}
```

Use the Read tool on the `path` value to view the captured image.

Panorama

Captures a 360-degree composite image: four screenshots (forward, right, backward, left) arranged in a 2x2 grid with directional labels.

```curl
curl -s -X POST http://localhost:5173/agent/panorama
```

Response:

```json
{
  "success": true,
  "path": "_agent-output/panorama-1708300000.png",
  "position": { "x": 0, "y": 1.4, "z": 0 },
  "orientation": { "yaw": 0, "pitch": 0 }
}
```

Surroundings

Raycasts in 10 directions and returns distance to the nearest surface in each. Returns -1 if nothing is hit within 100 units.

```curl
curl -s http://localhost:5173/agent/surroundings
```

Response:

```json
{
  "position": { "x": 0, "y": 1.4, "z": 0 },
  "orientation": { "yaw": 0, "pitch": 0 },
  "distances": {
```

```
    "forward": 5.2,
    "forward_right": 4.8,
    "right": 6.1,
    "back_right": 5.5,
    "back": 6.3,
    "back_left": 5.9,
    "left": 5.4,
    "forward_left": 4.6,
    "up": -1,
    "down": 1.2
  }
}
```

Raycast

Casts a ray from the player's eye position in one or more directions. Uses Rapier physics for accurate collision detection against level geometry.

```
# Single direction
curl -s -X POST http://localhost:5173/agent/raycast \
  -H "Content-Type: application/json" \
  -d '{"direction": "forward"}'

# Multiple directions
curl -s -X POST http://localhost:5173/agent/raycast \
  -H "Content-Type: application/json" \
  -d '{"directions": ["forward", "left", "right"]}'
```

Single response:

```
{
  "hit": true,
  "distance": 5.2,
  "hitPoint": { "x": 0, "y": 1.4, "z": 5.2 },
  "normal": { "x": 0, "y": 0, "z": -1 }
}
```

Valid directions: `forward`, `backward`, `left`, `right`, `up`, `down`, `forward_right`, `forward_left`, `back_right`, `back_left`.

Screen Raycast

Converts a pixel coordinate on the screenshot to a 3D world position. This is how the agent answers "what is at this pixel?" — it shoots a ray from the camera through that screen pixel into the physics world.

```
# What's at pixel (320, 240)?
curl -s -X POST http://localhost:5173/agent/screen-raycast \
  -H "Content-Type: application/json" \
  -d '{"screenX": 320, "screenY": 240}'
```

```
# Same, but also auto-rotate the camera to face the hit point
curl -s -X POST http://localhost:5173/agent/screen-raycast \
  -H "Content-Type: application/json" \
  -d '{"screenX": 320, "screenY": 240, "rotate": true}'
```

Response:

```
{
  "hit": true,
  "screenX": 320,
  "screenY": 240,
  "hitPoint": { "x": 2.5, "y": 0.5, "z": 8.3 },
  "distance": 8.9,
  "normal": { "x": 0.1, "y": 0.9, "z": 0 },
  "rotated": true,
  "orientation": { "yaw": 15.3, "pitch": -8.2 },
  "debugScreenshotPath": "_agent-output/debug-screen-raycast-1708300000.png"
}
```

The debug screenshot shows a green crosshair at the queried pixel.

Detect (DINO Object Detection)

Runs the Grounding DINO zero-shot object detection model on the most recently captured screenshot. Returns bounding boxes with pixel coordinates and confidence scores.

Requires the detect server to be running (`npm run detect-server`).

```
curl -s -X POST http://localhost:5173/agent/detect \
  -H "Content-Type: application/json" \
  -d '{"labels": "doorway . crate . wall . ramp . stairs", "threshold": 0.3}'
```

Response:

```
{
  "detections": [
    {
      "label": "doorway",
      "score": 0.95,
      "box": { "xmin": 150, "ymin": 100, "xmax": 250, "ymax": 300 },
      "center": { "x": 200, "y": 200 }
    },
    {
      "label": "crate",
      "score": 0.87,
      "box": { "xmin": 400, "ymin": 250, "xmax": 500, "ymax": 350 },
      "center": { "x": 450, "y": 300 }
    }
  ],
  "annotatedPath": "_agent-output/detect-1708300000.png",
  "count": 2
```

```
}
```

Labels are dot-separated, lowercase. The `annotatedPath` image shows colored bounding boxes drawn on the screenshot.

---

6. API Reference — Actions

Move

Walk in a direction relative to current facing. Left/right is strafing.

```
# Move forward 3 meters
curl -s -X POST http://localhost:5173/agent/move \
  -H "Content-Type: application/json" \
  -d '{"direction": "forward", "distance": 3}'

# Move until blocked (omit distance)
curl -s -X POST http://localhost:5173/agent/move \
  -H "Content-Type: application/json" \
  -d '{"direction": "right"}'

# Stop if elevation changes more than 1 unit (ramp/stair detection)
curl -s -X POST http://localhost:5173/agent/move \
  -H "Content-Type: application/json" \
  -d '{"direction": "forward", "distance": 10, "stopOnElevationChange": 1.0}'
```

Response:

```
{
  "success": true,
  "action": "move",
  "intended": { "direction": "forward", "distance": 3 },
  "actual": {
    "displacement": { "x": 0, "y": 0, "z": 2.95 },
    "distance": 2.95
  },
  "position": { "x": 0, "y": 1.4, "z": 2.95 },
  "orientation": { "yaw": 0, "pitch": 0 },
  "grounded": true,
  "events": [],
  "blocked": false,
  "surface": {
    "normal": { "x": 0, "y": 1, "z": 0 },
    "slopeAngle": 0
  }
}
```

Movement events that can appear in the `events` array:

Event	Meaning
`collision`	Hit something while moving
`blocked`	Movement stopped by an obstacle
`partial`	Only moved less than 95% of intended distance
`fell`	Started falling (went off an edge)
`landed`	Landed after a fall
`elevation_stop`	Y displacement exceeded the threshold
`cancelled`	Movement was cancelled via /agent/stop

Rotate

Instantly turn the camera. Yaw is horizontal, pitch is vertical.

```
curl -s -X POST http://localhost:5173/agent/rotate \
  -H "Content-Type: application/json" \
  -d '{"yaw": 90, "pitch": 0}'
```

- **Yaw:** Positive = turn left, negative = turn right (degrees)
- **Pitch:** Positive = look up, negative = look down, clamped to plus/minus 90 degrees

Jump

Jump in place. Must be grounded. Blocks until landing.

```
curl -s -X POST http://localhost:5173/agent/jump
```

Jump + Move

Jump while simultaneously moving in a direction. Jump velocity is 6 units/sec, horizontal speed is 8 units/sec.

```
curl -s -X POST http://localhost:5173/agent/jump-move \
  -H "Content-Type: application/json" \
  -d '{"direction": "forward", "distance": 3}'
```

Move To

Walk toward an absolute world XZ coordinate. The character preserves its current facing direction and walks directly toward the target. Completes within approximately 0.3 units of the target, or when blocked, or on timeout.

```
curl -s -X POST http://localhost:5173/agent/move-to \
  -H "Content-Type: application/json" \
  -d '{"x": 20, "z": 35}'
```

Look At

Rotate the camera to face a specific 3D world point.

```
curl -s -X POST http://localhost:5173/agent/look-at \
  -H "Content-Type: application/json" \
  -d '{"x": 16, "y": 3, "z": 30}'
```

Sequence

Execute multiple commands in one HTTP call. Commands run sequentially. Optionally stop the sequence if any movement command is blocked.

```
curl -s -X POST http://localhost:5173/agent/sequence \
  -H "Content-Type: application/json" \
  -d '{
    "actions": [
      {"type": "move", "direction": "forward", "distance": 5},
      {"type": "rotate", "yaw": -90},
      {"type": "move", "direction": "forward", "distance": 3}
    ],
    "stopOnBlocked": true
  }'
```

Teleport

Instantly move to a world coordinate (no physics, no collision).

```
curl -s -X POST http://localhost:5173/agent/teleport \
  -H "Content-Type: application/json" \
  -d '{"x": 10, "y": 1.6, "z": 30}'
```

Undo

Return to the position from N actions ago.

```
curl -s -X POST http://localhost:5173/agent/undo \
  -H "Content-Type: application/json" \
  -d '{"steps": 1}'
```

History

View the last 20 action snapshots with positions and timestamps.

```
curl -s http://localhost:5173/agent/history
```

Stop

Cancel any active movement command.

```
curl -s -X POST http://localhost:5173/agent/stop
```

---

7. DINO Object Detection

What It Is

Grounding DINO is a zero-shot object detection model. "Zero-shot" means it can detect objects described by arbitrary text labels without being trained on those specific objects. You tell it what to look for (e.g., "doorway", "crate", "ramp") and it returns bounding boxes with confidence scores.

The project includes a lightweight Python server (`detect-server.py`) that wraps the `IDEA-Research/grounding-dino-tiny` model behind a FastAPI endpoint.

How It Works

```
1. Agent takes a screenshot
      curl -s -X POST http://localhost:5173/agent/screenshot

2. Agent requests detection on that screenshot
      curl -s -X POST http://localhost:5173/agent/detect \
        -H "Content-Type: application/json" \
        -d '{"labels": "doorway . crate . wall"}'

3. Vite plugin reads the last screenshot path, sends it to detect-server.py

4. detect-server.py:
   a. Loads the image from disk
   b. Runs Grounding DINO inference (GPU or CPU)
   c. Extracts bounding boxes above the confidence threshold
   d. Draws annotated boxes on a copy of the image
   e. Returns detections + annotated image path

5. Agent receives bounding boxes with pixel coordinates and center points
```

The Detection Server

The server (`detect-server.py`) runs on `http://127.0.0.1:8190`:

- **Model:** `IDEA-Research/grounding-dino-tiny` (from Hugging Face)
- **Device:** CUDA GPU if available, otherwise CPU
- **Health check:** `GET /health`
- **Detection:** `POST /detect` with `image_path`, `labels`, and optional `threshold`

Label Format

Labels are dot-separated, lowercase strings:

```
"doorway . door . opening . corridor"
"crate . box . container"
"ramp . stairs . incline"
"wall . pillar . column"
```

The model works best with simple, descriptive 1-3 word labels. The default confidence threshold is 0.3 (30%). Lower values return more detections but with more false positives.

Annotated Output

Every detection call produces an annotated image saved to `_agent-output/detect-{timestamp}.png`. The image shows:

- Colored bounding boxes around each detection (cycling through green, red, blue, yellow, magenta, cyan, orange, purple)
- Label text and confidence score above each box

The Detection-to-Navigation Workflow

This is the key pattern that connects vision to movement:

```
Step 1: Screenshot
  curl -s -X POST http://localhost:5173/agent/screenshot

Step 2: Detect objects
  curl -s -X POST http://localhost:5173/agent/detect \
    -H "Content-Type: application/json" \
    -d '{"labels": "doorway . opening"}'

Step 3: Use the center pixel of a detection to aim and measure distance
  curl -s -X POST http://localhost:5173/agent/screen-raycast \
    -H "Content-Type: application/json" \
    -d '{"screenX": 200, "screenY": 200, "rotate": true}'

Step 4: Walk toward the target (distance minus safety margin)
  curl -s -X POST http://localhost:5173/agent/move \
    -H "Content-Type: application/json" \
    -d '{"direction": "forward", "distance": 4.5}'
```

This **screenshot, detect, screen-raycast, move** loop is the primary navigation technique. The DINO model provides semantic understanding ("that's a doorway"), screen-raycast converts the 2D pixel to a 3D world point and distance, and move walks the character there.

---

8. Navigation Patterns

Coordinate System

Understanding the coordinate system is essential for navigation:

- **Y is up.** The ground plane is XZ.
- **Yaw 0** means facing the -Z direction. Positive yaw = turn left.
- **Pitch 0** means looking level. Positive pitch = look up.
- **Position Y** includes eye height (~1.4m when on flat ground at Y=0).
- **Movement speed** is 8 units per second.

Room Entry Protocol

This is the core exploration loop. Follow it every time you enter a new area:

A. Panorama — Take a 360-degree panorama to survey all directions.

B. Assess — Analyze all four quadrants. Rank directions by interest: doorways and openings > objects and features > open corridors > blank walls.

C. Rotate — Face the most interesting direction.

D. Detect — Take a screenshot, then run DINO detection to find doorways, objects, and features with bounding boxes.

E. Aim — Use screen-raycast on the bounding box center (with `rotate: true`) to auto-face the target and measure its distance.

F. Move — Walk forward by the measured distance minus ~1 unit safety margin.

G. Document — Record what you did, saw, and plan to do next.

H. Repeat — You are in a new area. Start from step A.

Doorway Navigation (Detect-Aim-Walk)

The most reliable method for passing through doorways:

1. Take a screenshot facing the doorway direction
2. Run DINO detection: `{"labels": "doorway . door . door frame . opening"}`
3. Screen-raycast at the detection center with `"rotate": true`
4. Level pitch if needed: `{"yaw": 0, "pitch": 0}` (screen-raycast may tilt the view)
5. Move forward by the distance minus 1 unit

Wall Hugging

For following corridors:

1. Strafe to a wall: `{"direction": "right"}` (no distance = move until blocked)
2. Walk forward along the wall: `{"direction": "forward"}` (until blocked)
3. Check the Y value in the response — changes indicate ramps or stairs

Backtracking When Stuck

If `blocked: true` appears in a movement response:

1. Move backward 2-3 units
2. Take a panorama (not just a forward screenshot)
3. Pick the next most interesting direction
4. Never push against a wall repeatedly

Error Handling

HTTP Status	Meaning	What To Do
200	Success	Parse the JSON result
400	Bad request	Check JSON format and parameter values
409	Action in progress	Wait, or call /agent/stop first
502	Detection server error	Check that detect-server is running

HTTP Status	Meaning	What To Do
503	Browser not connected	Open `http://localhost:5173/?agent` in a browser
504	Command timed out	Movement took too long or got stuck

Response signals to watch for:

Signal	Meaning	Action
`"blocked": true`	Hit a wall	Do not retry same direction. Back up.
`"events": ["fell"]`	Went off an edge	Check new Y position.
`"actual.distance": 0`	Did not move at all	Stuck. Back up and reassess.
`"events": ["elevation_stop"]`	Significant height change	Ramp or stairs encountered.

---

9. Getting Started

Prerequisites

- **Node.js** (v18+) and npm
- **Python 3.10+** with pip (only needed for DINO detection)
- A modern browser (Chrome, Edge, Firefox)

Install Dependencies

```
# JavaScript dependencies
npm install

# Python dependencies (for object detection, optional)
pip install -r requirements.txt
```

Start the Servers

Terminal 1 — Dev server:

```
npm run dev
```

This starts Vite on `http://localhost:5173` with the agent plugin middleware.

Terminal 2 — Detection server (optional):

```
npm run detect-server
```

This starts Grounding DINO on `http://127.0.0.1:8190`. First run downloads the model weights (~350 MB).

Open the Browser

Open the 3D world in agent mode:

```
http://localhost:5173/?agent&level=glb
```

You should see the 3D world rendered in the browser. The console should show "Agent mode active, waiting for connection..." and then "Browser connected" when the WebSocket links up.

First Commands

Verify the connection:

```
curl -s http://localhost:5173/agent/state
```

Take your first look around:

```
curl -s -X POST http://localhost:5173/agent/panorama
```

The response includes a file path. Use Claude Code's Read tool on that path to view the 360-degree panorama image.

Move forward and see what happens:

```
curl -s -X POST http://localhost:5173/agent/move \
  -H "Content-Type: application/json" \
  -d '{"direction": "forward", "distance": 3}'
```

Take a screenshot and detect objects:

```
curl -s -X POST http://localhost:5173/agent/screenshot
curl -s -X POST http://localhost:5173/agent/detect \
  -H "Content-Type: application/json" \
  -d '{"labels": "doorway . wall . corridor . crate"}'
```

Quick Exploration Walkthrough

Here is a complete sequence that demonstrates the detect-aim-walk pattern:

```
# 1. Check current state
curl -s http://localhost:5173/agent/state

# 2. Survey the room
curl -s -X POST http://localhost:5173/agent/panorama

# 3. Take a forward screenshot
curl -s -X POST http://localhost:5173/agent/screenshot

# 4. Detect doorways in the screenshot
curl -s -X POST http://localhost:5173/agent/detect \
  -H "Content-Type: application/json" \
  -d '{"labels": "doorway . opening . corridor"}'

# 5. Aim at the first doorway's center pixel (from detection response)
```

```
#    and auto-rotate to face it
curl -s -X POST http://localhost:5173/agent/screen-raycast \
  -H "Content-Type: application/json" \
  -d '{"screenX": 200, "screenY": 200, "rotate": true}'

# 6. Walk toward the doorway (use distance from raycast minus 1 unit)
curl -s -X POST http://localhost:5173/agent/move \
  -H "Content-Type: application/json" \
  -d '{"direction": "forward", "distance": 4}'

# 7. You're in a new room - take a panorama and repeat
curl -s -X POST http://localhost:5173/agent/panorama
```

Appendix: Coordinate Quick Reference

Property	Convention
Up axis	Y
Ground plane	XZ
Yaw 0 degrees	Facing -Z
Yaw positive	Turn left (counter-clockwise from above)
Pitch 0 degrees	Looking level/horizontal
Pitch positive	Looking up
Pitch range	-90 to +90 degrees
Eye height	~0.7m above physics body
Standing Y on flat ground	~1.4m
Character capsule radius	0.3m
Character capsule height	0.6m
Movement speed	8 units/sec
Jump velocity	6 units/sec
Max ray distance	100 units

Appendix: Complete API Endpoint Table

Endpoint	Method	Parameters	Returns
`/agent/state`	GET	—	position, orientation, grounded, velocity
`/agent/screenshot`	POST	yaw, pitch, yawOffset, pitchOffset (all optional)	path to PNG file
`/agent/panorama`	POST	—	path to composite PNG
`/agent/surroundings`	GET	—	10-direction distance map

Endpoint	Method	Parameters	Returns
`/agent/raycast`	POST	direction or directions[]	hit, distance, hitPoint, normal
`/agent/screen-raycast`	POST	screenX, screenY, rotate (optional)	hitPoint, distance, orientation
`/agent/detect`	POST	labels, threshold (optional)	detections[], annotatedPath
`/agent/move`	POST	direction, distance (optional), stopOnElevationChange (optional)	displacement, blocked, events
`/agent/rotate`	POST	yaw, pitch	orientation
`/agent/jump`	POST	—	displacement, events
`/agent/jump-move`	POST	direction, distance (optional)	displacement, events
`/agent/move-to`	POST	x, z	displacement, blocked
`/agent/look-at`	POST	x, y, z	orientation
`/agent/sequence`	POST	actions[], stopOnBlocked (optional)	array of results
`/agent/teleport`	POST	x, y, z	position
`/agent/undo`	POST	steps (optional, default 1)	position
`/agent/history`	GET	—	entries[], count
`/agent/stop`	POST	—	success